راهنمای جامع GitHub



راهنمایی برای آشنایی با GitHub، زبان Markdown، همکاری در پروژه‌های نرم‌افزاری و ساخت یک پروفایل حرفه‌ای برای برنامه‌نویسان.

فهرست مطالب

- تاریخچه
- حساب کاربری GitHub
  - مخازن (Repositories)
  - مجوزهای نرم‌افزاری (Licenses)
  - فایل `.gitignore`
  - فایل `README.md`
- زبان Markdown
- نوشتن رزومه در GitHub

- GitHub Desktop
- سازمان‌ها در GitHub
- GitHub Pages
- Issues در GitHub
- شاخه‌ها (Branches)
- درخواست ادغام (Pull Requests)
- ادغام شاخه‌ها (Merge)
- فورک (Fork)
- جمع‌بندی
- منابع مفید

تاریخچه

برای اینکه بفهمیم یک فناوری چگونه کار می‌کند و چرا به وجود آمده است، ابتدا باید بدانیم چه مشکلی را حل می‌کند.

در حال توسعه هستهٔ لینوکس بود، مدیریت مشارکت (Linus Torvalds) زمانی که لینوس تورووالدز برنامه‌نویسان و پیگیری تغییرات در یک پروژهٔ بزرگ با مشکلات زیادی همراه بود. به همین دلیل، او در سال ۲۰۰۵ سیستم کنترل نسخه‌ای به نام **Git** را ایجاد کرد.

Git یک سیستم کنترل نسخهٔ توزیع‌شده (Distributed Version Control System) است که امکان ثبت تغییرات، مدیریت شاخه‌ها و همکاری هم‌زمان برنامه‌نویسان روی پروژه‌های نرم‌افزاری را فراهم می‌کند.

برای آشنایی بیشتر با Git، به راهنمای آموزش Git مراجعه کنید.

GitHub در سال ۲۰۰۸، پلتفرمی به نام Git بر پایهٔ راه‌اندازی شد. این پلتفرم به برنامه‌نویسان اجازه می‌دهد پروژه‌های خود را به‌صورت آنلاین میزبانی کنند، در پروژه‌های متن‌باز مشارکت داشته باشند، کدهای یکدیگر را بررسی کنند و با سایر توسعه‌دهندگان همکاری کنند.

در سال ۲۰۱۸، شرکت Microsoft، گیت‌هاب را خریداری کرد. این اتفاق به گسترش ارتباط میان GitHub و سرویس‌های Visual Studio Code، ابزارهای مایکروسافت، از جمله کمک کرد.

امروزه GitHub تنها محلی برای ذخیره‌سازی کدها نیست؛ بلکه بستری برای همکاری تیمی، توسعهٔ متن‌باز، مدیریت پروژه و معرفی توانایی‌های حرفه‌ای برنامه‌نویسان محسوب می‌شود.

حساب کاربری **GitHub**

با داشتن یک حساب کاربری GitHub می‌توانید مخازن خود را مدیریت کنید، در پروژه‌های مختلف مشارکت داشته باشید، تغییرات را پیگیری کنید و از ابزارهای همکاری این پلتفرم استفاده کنید.

ساخت حساب کاربری

برای ساخت حساب کاربری مراحل زیر را دنبال کنید:

1. وارد GitHub وب‌سایت شوید و روی گزینهٔ **Sign up** کلیک کنید.
2. آدرس ایمیل خود را وارد کنید و یک رمز عبور قوی انتخاب کنید.
3. نام کاربری خود را با دقت انتخاب کنید؛ زیرا بخشی از آدرس عمومی پروفایل شما خواهد بود:

   ```
   https://github.com/your-username
   ```

4. مراحل تأیید حساب کاربری را تکمیل کنید.
5. وارد پروفایل خود شوید و روی گزینهٔ **Edit profile** کلیک کنید.
6. موقعیت مکانی و سایر اطلاعات مرتبط را، تصویر پروفایل، نام نمایشی، توضیحات کوتاه (Bio) وارد کنید.
7. در صورت نیاز، نام دانشگاه، شرکت یا وب‌سایت شخصی خود را نیز اضافه کنید.

> نکته: نام کاربری خود را به‌گونه‌ای انتخاب کنید که حرفه‌ای، ساده و به‌یادماندنی باشد.

به‌طور معمول، هر آدرس ایمیل در یک زمان به یک حساب GitHub اختصاص داده می‌شود. البته می‌توانید ایمیل متصل به حساب خود را تغییر دهید.

مخازن (Repositories)

Repository یا به‌اختصار **Repo**، محلی برای نگهداری فایل‌های یک پروژه و پیگیری تغییرات آن‌هاست.

یک مخزن می‌تواند شامل کدهای برنامه‌نویسی، مستندات، تصاویر، فایل‌های پیکربندی و سایر منابع مرتبط با پروژه باشد.

GitHub تغییرات فایل‌ها را ثبت و مدیریت می‌کند و Git مخزن را به‌صورت آنلاین میزبانی می‌کند و ابزارهای لازم برای همکاری را در اختیار توسعه‌دهندگان قرار می‌دهد.

مخازن GitHub دو نوع اصلی دارند:

- **Public:** مخزن عمومی است و همه می‌توانند محتوای آن را مشاهده کنند.
- **Private:** مخزن خصوصی است و فقط افراد دارای مجوز به آن دسترسی دارند.

می‌توانید برای پروژه‌های شخصی، تکالیف دانشگاهی، مشارکت‌های متن‌باز یا ساخت رزومهٔ حرفه‌ای خود مخزن ایجاد کنید.

ساخت یک **Repository**

۱. وارد حساب GitHub خود شوید.

۲. روی علامت + کلیک کنید و گزینهٔ **New repository** را انتخاب کنید.

۳. نام مخزن و در صورت تمایل توضیحات آن را وارد کنید.

۴. مشخص کنید مخزن عمومی باشد یا خصوصی.

۵. یا مجوز نرم‌افزاری اضافه `.gitignore` قالب ، `README.md` در صورت نیاز، هنگام ساخت مخزن یک فایل کنید.

۶. روی **Create repository** کلیک کنید.

مجوزهای نرم‌افزاری (Licenses)

مجوز نرم‌افزاری مشخص می‌کند دیگران تحت چه شرایطی اجازه دارند از کدهای پروژهٔ شما استفاده کنند، آن‌ها را تغییر دهند، بازتوزیع کنند یا در پروژه‌های دیگر به کار ببرند.

انتخاب مجوز، به‌خصوص در پروژه‌های متن‌باز، اهمیت زیادی دارد؛ زیرا عمومی بودن یک مخزن به‌معنای اجازهٔ استفادهٔ آزاد از تمام کدهای آن نیست.

در ادامه با سه حالت رایج آشنا می‌شویم.

مجوز **MIT**

مجوز MIT یک مجوز متن‌باز با محدودیت‌های نسبتاً کم است.

این مجوز به دیگران اجازه می‌دهد از کد شما استفاده کنند، آن را تغییر دهند، بازتوزیع کنند و حتی در پروژه‌های تجاری به کار ببرند؛ به شرط آنکه اطلاعیه‌های مربوط به حق نشر و متن مجوز را حفظ کنند.

این مجوز برای پروژه‌های شخصی، کتابخانه‌های نرم‌افزاری و بسیاری از پروژه‌های متن‌باز انتخاب محبوبی است.

مجوز **Apache 2.0**

مجوز Apache 2.0 نیز یک مجوز متن‌باز با محدودیت‌های کم است و امکان استفاده، تغییر و بازتوزیع کد را فراهم می‌کند.

یکی از تفاوت‌های مهم آن با MIT، داشتن شرایط صریح‌تر دربارهٔ حقوق اختراع (Patent Rights) است. همچنین رعایت برخی الزامات مربوط به اطلاعیه‌ها و تغییرات ضروری است.

این مجوز برای پروژه‌هایی مناسب است که شفافیت دربارهٔ حقوق اختراع در آن‌ها اهمیت دارد.

بدون مجوز **(No License)**

اگر برای پروژهٔ خود مجوزی تعیین نکنید، کد شما عموماً تحت حمایت قوانین پیش‌فرض حق نشر قرار می‌گیرد.

دیگران ممکن است بتوانند مخزن عمومی شما را مشاهده کنند، اما صرفاً به دلیل عمومی بودن مخزن، اجازهٔ قانونی استفاده، کپی‌برداری، تغییر یا توزیع کد را به دست نمی‌آورند.

> پیشنهاد: اگر می‌خواهید دیگران بتوانند از پروژهٔ متن‌باز شما استفاده کنند یا در توسعهٔ آن مشارکت داشته باشند، مجوز مناسبی انتخاب کنید و پیش از انتشار، شرایط آن را بخوانید.

برای اطلاعات بیشتر به وب‌سایت Choose a License مراجعه کنید.

فایل `.gitignore`

فایل `.gitignore` به Git می‌گوید کدام فایل‌ها و پوشه‌های هنوز ثبت‌نشده را نادیده بگیرد.

هنگام توسعهٔ نرم‌افزار، فایل‌های زیادی ایجاد می‌شوند که لزوماً نباید وارد مخزن شوند. برای مثال:

- فایل‌های موقت
- خروجی‌های حاصل از Build
- پوشه‌های مربوط به کتابخانه‌ها و وابستگی‌ها
- فایل‌های تنظیمات شخصی
- فایل‌های محیطی و اطلاعات حساس

فایل `.gitignore` کمک می‌کند این موارد به‌صورت ناخواسته به مخزن اضافه نشوند.

یک مثال کاربردی

```
# وابستگی‌ها
node_modules/

# خروجی‌های Build
dist/
build/

# متغیرهای محیطی و تنظیمات محرمانه
.env
.env.local

# فایل‌های موقت پایتون
__pycache__/
*.py[cod]

# محیط‌های مجازی پایتون
.venv/
venv/

# فایل‌های سیستم‌عامل
.DS_Store
Thumbs.db
```

```
# تنظیمات و فایل‌های ویرایشگر
.vscode/
```

الگوهای رایج در `.gitignore`

الگو	کاربرد
`file.txt`	نادیده گرفتن فایل مطابق با این مسیر یا نام
`*.log`	نادیده گرفتن فایل‌هایی که پسوند `.log` دارند
`temp/`	نادیده گرفتن پوشه‌ای به نام `temp`
`!important.log`	خارج کردن فایل مطابق با این الگو از قوانین نادیده‌گیری
`/config.json`	نادیده گرفتن `config.json` در ریشهٔ مخزن
`**/cache/`	نادیده گرفتن پوشه‌هایی با نام `cache` در مسیرهای مختلف

نکات مهم دربارهٔ `.gitignore`

- فایل‌هایی را که قبلاً تحت پیگیری Git (Tracked) قرار نگرفته‌اند و با الگوهای این فایل مطابقت دارند، نادیده می‌گیرد.
- اگر فایلی قبلاً به Commit اضافه و Git شده باشد، افزودن نام آن به `.gitignore` باعث حذف خودکار آن از مخزن نمی‌شود.
- هرگز رمز عبور، کلید API، Access Token یا اطلاعات محرمانه را در مخزن ثبت نکنید.
- اگر اطلاعات حساسی را قبلاً به مخزن آنلاین ارسال کرده‌اید، صرفاً حذف فایل یا افزودن آن به `.gitignore` کافی نیست. باید کلید یا رمز افشاشده را از سرویس مربوطه باطل یا تعویض کنید.

برای استفاده از قالب‌های آمادهٔ `.gitignore` می‌توانید به مخزن رسمی قالب‌های GitHub مراجعه کنید.

فایل `README.md`

فایل `README.md` معمولاً اولین بخشی است که بازدیدکنندگان هنگام ورود به یک مخزن مشاهده می‌کنند.

این فایل برای معرفی پروژه، توضیح کاربرد آن و راهنمایی کاربران و توسعه‌دهندگان نوشته می‌شود.

یک README مناسب می‌تواند شامل بخش‌های زیر باشد:

- نام و توضیحات پروژه
- قابلیت‌ها و ویژگی‌ها
- تصاویر و ویدئوهای نمایشی
- راهنمای نصب
- مثال‌های استفاده

- فناوری‌های استفاده‌شده
- ساختار پوشه‌ها و فایل‌ها
- راهنمای مشارکت در پروژه
- اطلاعات مجوز نرم‌افزاری
- اطلاعات تماس

است؛ زبانی ساده برای قالب‌بندی متن که در ادامه با آن آشنا می‌شویم Markdown مخفف `.md` پسوند.

واضح و کامل کمک می‌کند دیگران در مدت کوتاهی متوجه شوند پروژهٔ شما چه کاری انجام README یک می‌دهد و چگونه می‌توانند از آن استفاده کنند.

زبان Markdown

با مشارکت آرون شوارتز (John Gruber) یک زبان نشانه‌گذاری سبک است که جان گروبر Markdown توسعه داد و در سال ۲۰۰۴ منتشر کرد (Aaron Swartz).

هدف Markdown این بود که نوشتن متن‌های قالب‌بندی‌شده ساده‌تر و خواناتر شود، بدون اینکه نویسنده استفاده کند HTML مجبور باشد برای هر تغییر ظاهری از تگ‌های طولانی.

را می‌توان به Markdown تبدیل کرد و از آن در مستندات، فایل‌های HTML بخش README، Issues، درخواست‌های Pull Request و سایر نوشته‌های فنی استفاده کرد.

را پشتیبانی می‌کند که با نام Markdown نسخهٔ گسترش‌یافته‌ای از GitHub **GitHub Flavored Markdown (GFM)** شناخته می‌شود.

دستورات پایهٔ Markdown

۱. عنوان‌ها (Headings)

برای ایجاد عنوان از علامت `#` استفاده می‌کنیم. تعداد علامت‌های `#` سطح عنوان را مشخص می‌کند.

```
# عنوان سطح اول
## عنوان سطح دوم
### عنوان سطح سوم
#### عنوان سطح چهارم
##### عنوان سطح پنجم
###### عنوان سطح ششم
```

هرچه تعداد `#` بیشتر باشد، سطح عنوان پایین‌تر است.

بهتر است عنوان‌ها را به‌ترتیب منطقی استفاده کنید تا ساختار مستندات شما منظم و قابل‌فهم باشد.

۲. قالب‌بندی متن

متن ضخیم **(Bold)**:

```
**این متن ضخیم است**
__این متن نیز ضخیم است__
```

متن مورب (Italic):

```
*این متن مورب است*
_این متن نیز مورب است_
```

متن ضخیم و مورب:

```
***این متن هم ضخیم است و هم مورب***
```

~~متن خط‌خورده (Strikethrough)~~:

```
~~این متن خط خورده است~~
```

۳. لینک‌ها (Links)

برای ایجاد لینک از ساختار زیر استفاده می‌کنیم:

```
[گیت‌هاب](https://github.com/)
```

با کلیک روی عبارت «گیت‌هاب»، کاربر به آدرس مشخص‌شده منتقل می‌شود.

همچنین می‌توانید به بخش خاصی از همان فایل Markdown لینک بدهید:

```
[رفتن به بخش تاریخچه](#تاریخچه)
```

در GitHub، شناسهٔ لینک داخلی بر اساس عنوان بخش ساخته می‌شود؛ در عنوان‌های فارسی بهتر است پس از رندر شدن، عملکرد لینک را بررسی کنید.

۴. تصاویر (Images)

برای نمایش تصویر از علامت `!` ، متن جایگزین و آدرس تصویر استفاده می‌کنیم:

```
![لوگوی GitHub](https://github.githubassets.com/images/modules/logos_page/GitHub-Mark.png)
```

متن جایگزین (Alt Text) توضیحی کوتاه دربارهٔ تصویر است که در صورت بارگذاری نشدن تصویر نمایش داده می‌شود و به دسترس‌پذیری محتوا کمک می‌کند.

همچنین می‌توانید تصویر را به یک لینک تبدیل کنید:

```
[![لوگوی GitHub](https://github.githubassets.com/images/modules/logos_page/GitHub-Mark.png)](htt
```

در این حالت با کلیک روی تصویر، کاربر به وب‌سایت GitHub منتقل می‌شود.

۵. فهرست‌ها (Lists)

فهرست نامرتب (Unordered List)

برای ساخت فهرست نامرتب می‌توانید از `-`، `*` یا `+` استفاده کنید.

```
- مورد اول
- مورد دوم
- مورد سوم
```

فهرست مرتب (Ordered List)

برای ساخت فهرست شماره‌گذاری‌شده از عدد و نقطه استفاده می‌کنیم:

```
1. مورد اول
2. مورد دوم
3. مورد سوم
```

فهرست تو در تو (Nested List)

برای قرار دادن یک فهرست داخل فهرست دیگر، زیرمجموعه‌ها را با فاصله تورفتگی می‌دهیم:

```
- زبان‌های برنامه‌نویسی
  - Python
  - JavaScript
  - C++
- ابزارها
  - Git
  - GitHub
```

این ساختار برای دسته‌بندی مهارت‌ها، مراحل نصب و موضوعات آموزشی بسیار کاربردی است.

۶. خط جداکننده (Horizontal Rule)

برای ایجاد یک خط افقی بین بخش‌های متن، از سه یا چند خط تیره استفاده کنید:

```
---
```

این دستور برای جدا کردن بخش‌های اصلی یک مستند کاربرد دارد.

۷. نقل‌قول (Blockquote)

برای مشخص کردن نقل‌قول، یادداشت یا توضیحات ویژه از علامت `>` استفاده می‌کنیم:

```
> این یک نقل‌قول است.
>
> این قسمت پاراگراف دوم نقل‌قول است.
```

می‌توانید از این قابلیت برای برجسته کردن نکات مهم در راهنماها استفاده کنید.

۸. کد درون‌خطی و بلوک کد (Code)

برای نمایش یک دستور یا قطعه کد کوتاه درون متن، از یک جفت بک‌تیک استفاده می‌کنیم:

```
برای بررسی وضعیت مخزن از دستور `git status` استفاده کنید.
```

برای نمایش یک بلوک کد، سه بک‌تیک در ابتدا و انتهای کد قرار می‌دهیم:

````
```python
print("Hello, World!")
```
````

با نوشتن نام زبان بعد از بک‌تیک‌های ابتدایی، می‌توانید از قابلیت Syntax Highlighting استفاده کنید.

برای مثال:

````
```javascript
console.log("Hello, World!");
```
````

این قابلیت باعث می‌شود دستورات و اجزای مختلف کد با رنگ‌های متفاوت نمایش داده شوند.

۹. جدول‌ها (Tables)

برای ساخت جدول از علامت `|` برای جدا کردن ستون‌ها و از `-` برای تعریف ردیف عنوان استفاده می‌کنیم:

```
| نام | نوع | سطح |
|---|---|---|
| زبان برنامه‌نویسی | متوسط | Python |
| زبان نشانه‌گذاری | مقدماتی | HTML |
| زبان استایل‌دهی | مقدماتی | CSS |
```

همچنین می‌توانید نحوهٔ تراز شدن محتوای ستون‌ها را مشخص کنید:

```
| چپ‌چین | وسط‌چین | راست‌چین |
|:---|:---:|---:|
| A | B | C |
| D | E | F |
```

۱۰. فهرست کارها (Task Lists)

GitHub در Issues، Pull Requests و از فهرست‌های وظایف پشتیبانی می‌کند. این فهرست‌ها به‌خصوص در مستندات پروژه کاربرد دارند.

```
- [x] یادگیری مبانی Git
- [x] ساخت حساب GitHub
- [ ] انتشار یک پروژه
- [ ] مشارکت در یک پروژهٔ متن‌باز
```

- `[x]` نشان‌دهندهٔ وظیفهٔ انجام‌شده است.
- `[ ]` نشان‌دهندهٔ وظیفهٔ انجام‌نشده است.

۱۱. نمایش کاراکترهای ویژه

اگر بخواهید یک کاراکتر Markdown را به‌صورت عادی نمایش دهید و نگذارید به‌عنوان دستور قالب‌بندی تفسیر شود، می‌توانید قبل از آن از بک‌اسلش `\` استفاده کنید.

برای مثال:

```
\*این متن به‌صورت مورب نمایش داده نمی‌شود*\
```

این قابلیت هنگام آموزش یا مستندسازی دستورات Markdown بسیار کاربردی است.

۱۲. استفاده از HTML در Markdown

GitHub در فایل‌های Markdown اجازه می‌دهد از برخی تگ‌های HTML استفاده کنید.

برای مثال:

```
<p align="center">
  <strong>من خوش آمدید به GitHub پروفایل!</strong>
</p>
```

با این روش می‌توانید در برخی قسمت‌ها ظاهر محتوا را تنظیم کنید.

برای حفظ امنیت GitHub، HTML پشتیبانی نمی‌شوند. بااین‌حال، همهٔ تگ‌ها و ویژگی‌های رندرشده را پاک‌سازی می‌کند و ممکن است بعضی از تگ‌ها، ویژگی‌ها یا تنظیمات ظاهری را حذف یا

محدود کند.

نکته: تا جای ممکن از Markdown استاندارد استفاده کنید؛ زیرا خواناتر است و نگهداری آن آسان‌تر خواهد بود.

مراجعه کنید GitHub Flavored Markdown برای یادگیری بیشتر به مستندات.



نوشتن رزومه در GitHub

یکی از قابلیت‌های GitHub، ساخت یک فایل README اختصاصی برای پروفایل شخصی است. این فایل در صفحهٔ پروفایل شما نمایش داده می‌شود و می‌تواند مانند یک رزومهٔ کوتاه و حرفه‌ای عمل کند.

با استفاده از این قابلیت می‌توانید خودتان را معرفی کنید، مهارت‌هایتان را نشان دهید، پروژه‌های خود را به نمایش بگذارید و راه‌های ارتباطی حرفه‌ای خود را در اختیار دیگران قرار دهید.

ساخت Profile README

۱. یک مخزن جدید ایجاد کنید.

۲. نام مخزن را دقیقاً برابر با نام کاربری GitHub خود قرار دهید.

۳. مخزن را عمومی (Public) کنید.

۴. یک فایل با نام `README.md` در مخزن ایجاد کنید.

۵. اطلاعات مربوط به معرفی، مهارت‌ها، پروژه‌ها، دستاوردها و راه‌های ارتباطی خود را بنویسید.

۶. تغییرات را Commit کنید و سپس وارد صفحهٔ پروفایل خود شوید.

اگر نام مخزن دقیقاً با نام کاربری شما یکسان باشد، GitHub محتوای README شما را در پروفایل شما نمایش می‌دهد.

برای مثال، اگر نام کاربری شما `example-user` باشد، نام مخزن باید `example-user` باشد.

ساختار پیشنهادی رزومه

۱. معرفی (Introduction)

در ابتدای رزومه، خودتان را به‌صورت کوتاه معرفی کنید.

می‌توانید به علایق حرفه‌ای، موقعیت شغلی یا تحصیلی فعلی و حوزه‌هایی که دوست دارید در آن‌ها فعالیت کنید اشاره کنید.

۲. مهارت‌ها (Skills)

مهارت‌های خود را در دسته‌بندی‌های مشخص قرار دهید.

- زبان‌های برنامه‌نویسی: Python، JavaScript، C++، Java
- فریم‌ورک‌ها و کتابخانه‌ها: React، Django، Express
- ابزارها و پلتفرم‌ها: Git، GitHub، Docker، Linux
- پایگاه‌های داده: PostgreSQL، MySQL، MongoDB
- مهارت‌های حرفه‌ای: ارتباط مؤثر، کار تیمی، حل مسئله

فقط مهارت‌هایی را ذکر کنید که واقعاً با آن‌ها آشنا هستید و در صورت نیاز می‌توانید توانایی خود را نشان دهید.

۳. مدارک و دستاوردها (Certificates & Achievements)

مدارک معتبر، گواهی‌نامه‌ها، دوره‌های آموزشی، جوایز و سایر دستاوردهای مرتبط خود را فهرست کنید.

در صورت امکان، لینک مستقیم به مدرک یا صفحهٔ رسمی تأیید اعتبار آن اضافه کنید.

۴. نمونه‌کارها و پروژه‌ها (Portfolio & Projects)

پروژه‌هایی را معرفی کنید که توانایی‌های شما را به‌خوبی نشان می‌دهند.

برای هر پروژه، موارد زیر را بنویسید:

- نام پروژه و توضیح کوتاه
- فناوری‌های استفاده‌شده
- قابلیت‌های اصلی
- لینک مخزن کد
- در صورت وجود، لینک نسخهٔ آنلاین یا Demo

بهتر است به‌جای فهرست طولانی از مخازن ناقص، چند پروژهٔ کامل و کاربردی را معرفی کنید.

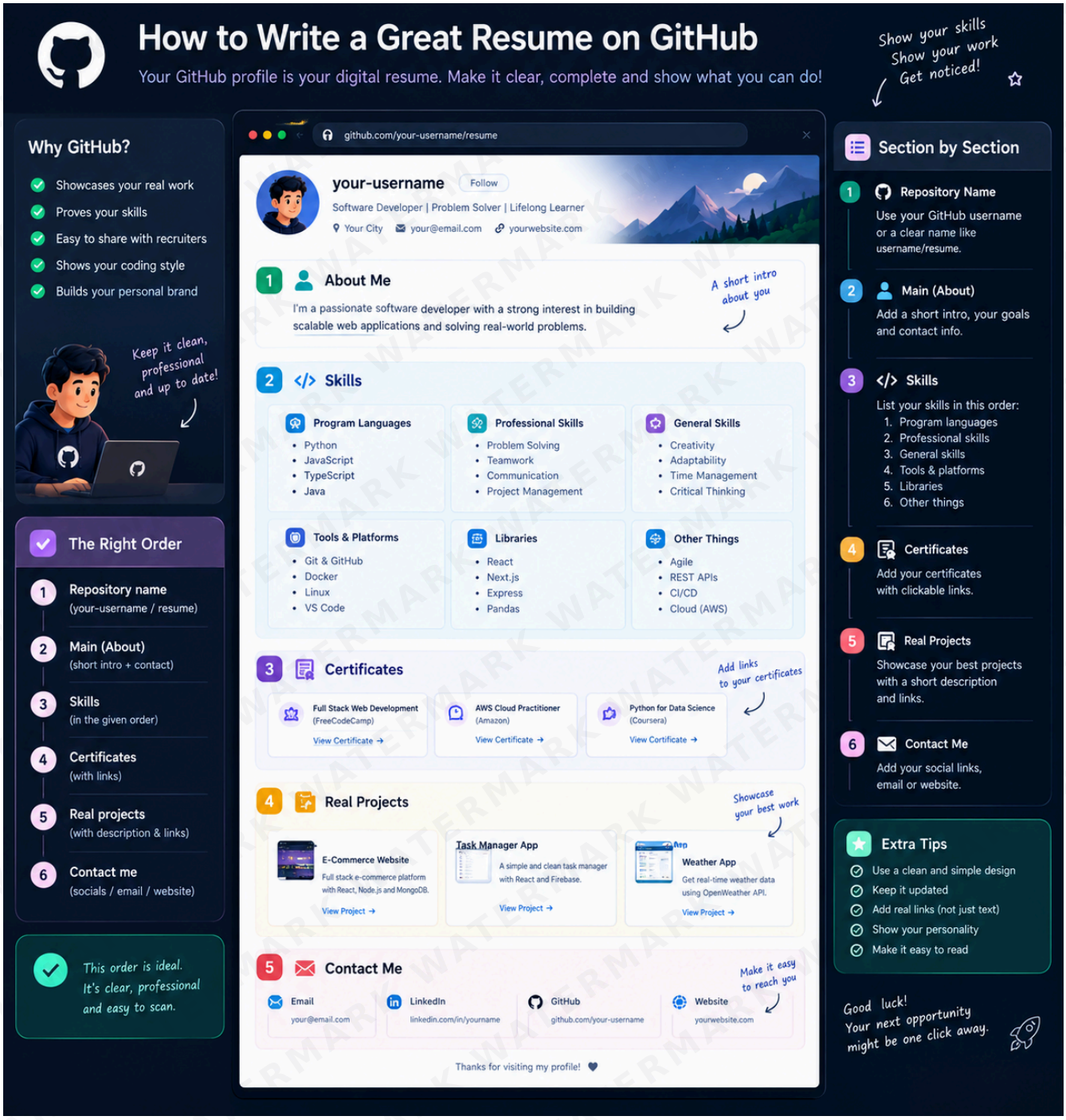
۵. اطلاعات تماس (Contact Information)

راه‌های ارتباطی حرفه‌ای خود را اضافه کنید؛ برای مثال:

- ایمیل
- لینک LinkedIn
- وب‌سایت شخصی
- پورتفولیو
- سایر پروفایل‌های حرفه‌ای مرتبط

از انتشار اطلاعات شخصی و حساس خودداری کنید.

> نکته: فایل Profile README را مختصر، منظم و به‌روز نگه دارید. مهم‌ترین مهارت‌ها و بهترین پروژه‌های شما باید به‌راحتی قابل مشاهده باشند.



GitHub Desktop

GitHub Desktop یک نرم‌افزار گرافیکی برای کار با مخازن Git است که به شما اجازه می‌دهد بسیاری از عملیات رایج Git را بدون نیاز به نوشتن دستورات خط فرمان انجام دهید.

این برنامه رابطی تصویری برای مشاهدهٔ تغییرات، ایجاد Commit، مدیریت Branch و همگام‌سازی پروژهٔ محلی با GitHub فراهم می‌کند.

نصب GitHub Desktop

۱. GitHub Desktop برنامه را از وب‌سایت رسمی دانلود کنید.

۲. آن را نصب و اجرا کنید.

۳. خود وارد شوید GitHub با حساب.

۴. کنید یا یک مخزن جدید بسازید Clone یک مخزن موجود را.

Clone کردن یک Repository

روی کامپیوتر خود، مراحل زیر را انجام دهید GitHub برای دانلود یک مخزن از:

۱. برنامهٔ GitHub Desktop را باز کنید.

۲. از منوی برنامه گزینهٔ **File → Clone Repository** را انتخاب کنید.

۳. مخزن موردنظر را از حساب خود انتخاب کنید یا آدرس آن را وارد کنید.

۴. پوشه‌ای را برای ذخیره‌سازی مخزن روی کامپیوتر انتخاب کنید.

۵. روی **Clone** کلیک کنید.

آن روی کامپیوتر شما دانلود می‌شود و می‌توانید به‌صورت محلی Git با این کار، مخزن به همراه تاریخچهٔ روی پروژه کار کنید.

اعمال و انتشار تغییرات

کردن مخزن Clone بعد از:

۱. پوشهٔ محلی پروژه را با ویرایشگری مانند Visual Studio Code باز کنید.

۲. فایل‌های موردنظر را ایجاد، ویرایش یا حذف کنید.

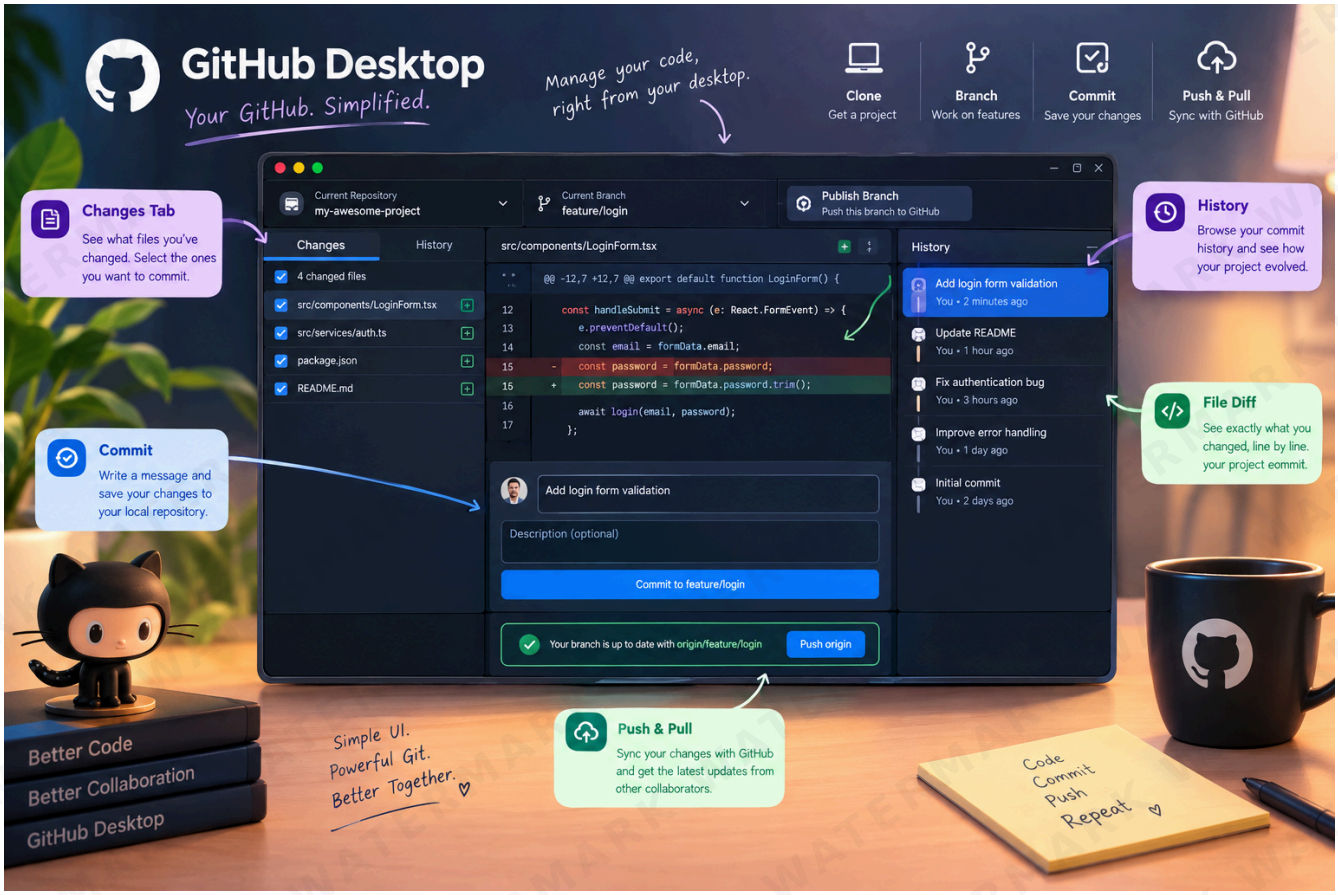
۳. به GitHub Desktop برگردید و تغییرات را بررسی کنید.

۴. یک پیام کوتاه و واضح برای Commit بنویسید.

۵. روی **Commit to main** یا گزینهٔ Commit مربوط به شاخهٔ فعلی کلیک کنید.

۶. روی **Push origin** کلیک کنید تا Commitهای محلی شما به GitHub ارسال شوند.

اگر مخزن هنوز روی GitHub منتشر نشده باشد، ممکن است به‌جای Push با گزینه‌ای مانند **Publish repository** مواجه شوید.



آشنایی با اصطلاحات اصلی

Clone

آن را نیز Git روی کامپیوتر شما دریافت می‌کند و تاریخچهٔ GitHub، مخزن را از یک محل راه دور، مانند همراه با فایل‌ها دانلود می‌کند.

Commit

ذخیره می‌کند Git یا نقطهٔ ثبت‌شده از تغییرات آمادهٔ ثبت را در مخزن محلی یک Snapshot.

مانند ثبت یک مرحلهٔ مشخص از پیشرفت پروژه است؛ با این تفاوت که Commit، به زبان ساده تاریخچه‌ای از تغییرات نیز ایجاد می‌شود.

Push

ارسال می‌کند، GitHub های محلی را به مخزن راه دور، مانند مخزن میزبانی‌شده درCommit.

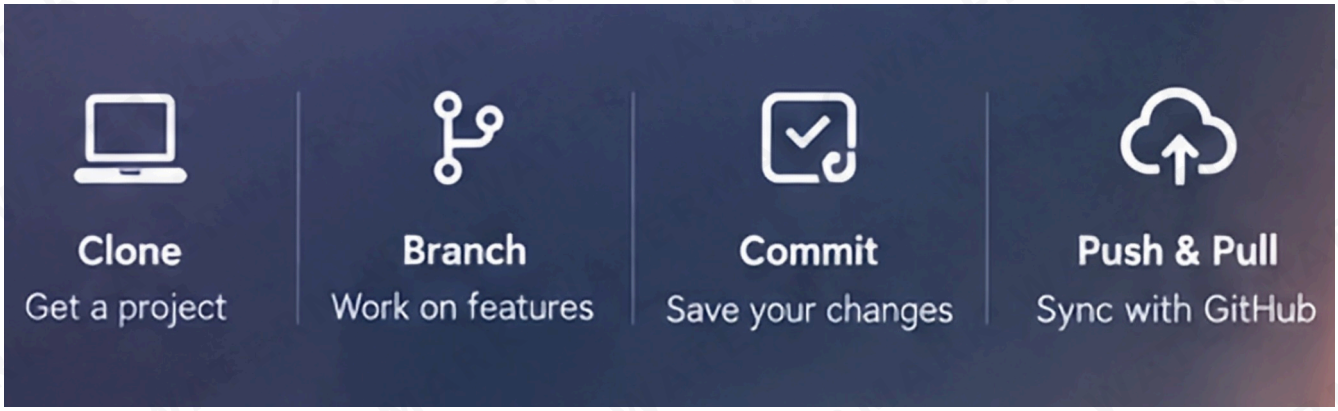
Pull

تغییرات مخزن راه دور را دریافت می‌کند و آن‌ها را با شاخهٔ محلی فعلی ادغام می‌کند.

Fetch

اطلاعات مربوط به تغییرات مخزن راه دور را دریافت می‌کند، اما به‌صورت خودکار آن تغییرات را وارد شاخهٔ فعلی نمی‌کند.

> نکتهٔ مهم: اجرای Commit به‌تنهایی تغییرات را به GitHub ارسال نمی‌کند. برای انتشار Commitها را در مخزن آنلاین، معمولاً باید آن‌ها را Push کنید.



سازمان‌ها در GitHub

ساختاری برای مدیریت مشترک حساب‌ها و مخازن است که برای شرکت‌ها، **GitHub Organization** استارتاپ‌ها، گروه‌های آموزشی و تیم‌های توسعه طراحی شده است.

با استفاده از سازمان‌ها می‌توان مخازن را مدیریت کرد، سطح دسترسی اعضا را تعیین کرد، مسئولیت‌ها را مشخص کرد و همکاری میان چندین توسعه‌دهنده را ساده‌تر کرد.

ساخت یک Organization

۱. وارد حساب GitHub خود شوید.

۲. از منوی حساب کاربری وارد بخش تنظیمات مربوط به سازمان‌ها شوید.

۳. گزینهٔ **New organization** را انتخاب کنید.

۴. طرح اشتراک متناسب با نیاز خود را انتخاب کنید.

۵. اطلاعات موردنیاز سازمان را وارد کنید.

۶. اعضای تیم را دعوت کنید و سطح دسترسی آن‌ها به مخازن را تنظیم کنید.

اگر امکانات طرح رایگان نیازهای شما را پوشش می‌دهد، می‌توانید کار را با همان طرح آغاز کنید.

نقش‌های اعضای سازمان

نقش‌های سازمان مشخص می‌کنند هر عضو چه اختیاراتی در سازمان داشته باشد.

نقش‌های رایج عبارت‌اند از:

- **Owner:** دسترسی مدیریتی گسترده به سازمان و تنظیمات آن دارد.
- **Member:** عضو عادی سازمان است و بر اساس مجوزهای تعیین‌شده به منابع دسترسی دارد.
- **Outside Collaborator:** می‌تواند به مخازن مشخصی دسترسی داشته باشد، بدون اینکه عضو سازمان محسوب شود.

در دسترس GitHub نقش‌ها و مجوزهای دیگری نیز ممکن است بر اساس تنظیمات سازمان و طرح باشند.

همیشه تلاش کنید هر فرد فقط مجوزهایی را داشته باشد که برای انجام وظایف خود به آن‌ها نیاز دارد.

ساخت **Profile README** برای سازمان

می‌توانید برای صفحهٔ عمومی سازمان نیز یک README اختصاصی ایجاد کنید.

مراحل انجام کار:

۱. در سازمان خود یک مخزن عمومی با نام `.github` بسازید.

۲. داخل این مخزن، پوشه‌ای با نام `profile` ایجاد کنید.

۳. درون پوشهٔ `profile` فایلی به نام `README.md` قرار دهید.

۴. در این فایل، سازمان، اهداف، پروژه‌ها و راهنمای مشارکت را معرفی کنید.

ساختار فایل‌ها باید به این صورت باشد:

```
.github/
└── profile/
    └── README.md
```

GitHub می‌تواند محتوای این README را در صفحهٔ عمومی سازمان نمایش دهد.



GitHub Pages

GitHub Pages یک سرویس میزبانی وب است که به شما اجازه می‌دهد وب‌سایت‌های ایستا (Static Websites) را مستقیماً از یک مخزن GitHub منتشر کنید.

این سرویس برای ساخت وب‌سایت شخصی، رزومهٔ آنلاین، پورتفولیو، مستندات پروژه و صفحات معرفی محصولات بسیار کاربردی است.

GitHub Pages عمدتاً از محتوای ایستا مانند HTML، CSS و JavaScript پشتیبانی می‌کند و به‌طور مستقیم برای اجرای برنامه‌های سمت سرور، مانند یک Backend مبتنی بر Node.js، PHP یا معمولی طراحی نشده است.

ساخت وب‌سایت با GitHub Pages

۱. یک مخزن در GitHub ایجاد کنید.

۲. فایل‌های وب‌سایت خود، از جمله `index.html` ، را در مخزن قرار دهید.

۳. وارد بخش **Settings** مخزن شوید.

۴. از منوی کناری، گزینهٔ **Pages** را انتخاب کنید.

۵. در بخش Build and deployment، روش انتشار مناسب را انتخاب کنید؛ برای مثال، انتشار از یک Branch و پوشهٔ مشخص یا استفاده از GitHub Actions.

۶. تنظیمات را ذخیره کنید و منتظر بمانید تا وب‌سایت منتشر شود.

آشنایی با آدرس‌های GitHub Pages

معمولاً در دو دسته قرار می‌گیرند GitHub Pages وب‌سایت‌های.

۱. وب‌سایت شخصی یا سازمانی

نام مخزن باید دقیقاً به این شکل باشد:

```
your-username.github.io
```

آدرس وب‌سایت معمولاً به شکل زیر خواهد بود:

```
https://your-username.github.io/
```

به‌جای `your-username` نام کاربری GitHub خود را قرار دهید.

۲. وب‌سایت مربوط به یک پروژه

می‌توانید نام عادی برای مخزن انتخاب کنید؛ برای مثال:

```
my-project
```

آدرس وب‌سایت معمولاً به این شکل خواهد بود:

```
https://your-username.github.io/my-project/
```

در این آدرس، نام کاربری و نام مخزن خود را جایگزین مقادیر نمونه کنید.

> نکته: آدرس نهایی به حساب کاربری یا سازمان، نام مخزن و تنظیمات انتشار بستگی دارد. همچنین می‌توانید از دامنهٔ اختصاصی خود استفاده کنید.

گزینه‌ای مناسب برای انتشار پورتفولیو و وب‌سایت‌های شخصی است؛ بدون اینکه نیازی به مدیریت یک سرور وب مستقل داشته باشید GitHub Pages.



GitHub در Issues

ابزار **GitHub Issues** برای گزارش باگ‌ها، پیشنهاد قابلیت‌های جدید، طرح پرسش و مدیریت وظایف مرتبط با یک مخزن استفاده می‌شود.

Issue صرفاً یک نظر یا کامنت معمولی نیست؛ بلکه موضوع یا وظیفه‌ای است که می‌توان آن را پیگیری کرد، به افراد اختصاص داد، برچسب‌گذاری کرد و دربارهٔ آن گفت‌وگو کرد.

کاربردهای رایج **Issues**

- **Bug Reports:** گزارش خطاها و رفتارهای غیرمنتظرهٔ نرم‌افزار
- **Feature Requests:** پیشنهاد قابلیت‌های جدید
- **Questions:** پرسیدن سؤال دربارهٔ پروژه
- **Task Tracking:** برنامه‌ریزی و پیگیری وظایف

چگونه یک **Issue** مناسب بنویسیم؟

هنگام گزارش یک باگ، توضیحات باید دقیق، روشن و تا حد امکان قابل تکرار باشند.

یک گزارش باگ مناسب بهتر است شامل موارد زیر باشد:

۱. **Title:** عنوانی کوتاه و مشخص دربارهٔ مشکل

۲. **Description:** توضیح مشکل و شرایطی که در آن رخ می‌دهد

۳. **Steps to Reproduce:** مراحل لازم برای بازتولید مشکل

۴. **Expected Behavior:** رفتاری که انتظار داشتید اتفاق بیفتد

۵. **Actual Behavior:** رفتاری که واقعاً اتفاق افتاده است

۶. **Environment:** اطلاعات مرتبط مانند سیستم‌عامل، نسخهٔ نرم‌افزار و محیط اجرا

در درخواست قابلیت جدید نیز بهتر است مشکلی را که قرار است حل شود توضیح دهید و مشخص کنید قابلیت پیشنهادی چه فایده‌ای دارد.

GitHub Issues از Markdown پشتیبانی می‌کند؛ بنابراین می‌توانید از عنوان‌ها، فهرست‌ها، لینک‌ها، بلوک‌های کد و فهرست وظایف برای خواناتر کردن توضیحات استفاده کنید.

> نکته: هنگام نوشتن Issue محترمانه، حرفه‌ای و دقیق باشید. یک گزارش واضح می‌تواند زمان زیادی را برای اعضای پروژه ذخیره کند.

شاخه‌ها (Branches)

Branch در Git یا شاخه، یک مسیر مستقل برای توسعهٔ نرم‌افزار درون یک مخزن است.

شاخه‌ها به برنامه‌نویسان اجازه می‌دهند قابلیت‌های جدیدی ایجاد کنند، باگ‌ها را برطرف کنند یا تغییرات آزمایشی انجام دهند؛ بدون اینکه مجبور باشند بلافاصله شاخهٔ اصلی پروژه را تغییر دهند.

شاخهٔ اصلی در بسیاری از مخازن `main` نام دارد؛ البته ممکن است در بعضی پروژه‌ها از نام‌های دیگری استفاده شود.

چرا از **Branch** استفاده می‌کنیم؟

فرض کنید یک تیم در حال توسعهٔ یک وب‌سایت است. یک برنامه‌نویس روی سیستم ورود کاربران کار می‌کند و برنامه‌نویس دیگری در حال رفع مشکل منوی سایت است.

اگر هرکدام روی شاخه‌ای جداگانه کار کنند، می‌توانند تغییرات خود را مستقل از یکدیگر توسعه دهند و آزمایش کنند.

برای مثال، ساختار شاخه‌ها می‌تواند به شکل زیر باشد:

```
main
├── feature/login
└── fix/navigation
```

هر شاخه می‌تواند Commitهای مخصوص به خود را داشته باشد. پس از بررسی و تأیید تغییرات، می‌توان آن‌ها را با شاخهٔ مقصد ادغام کرد.

ساخت یک Branch

می‌توانید با GitHub Desktop یا خط فرمان Git شاخهٔ جدیدی ایجاد کنید.

برای مثال، دستور زیر یک شاخه با نام `feature/login` ایجاد می‌کند و وارد آن می‌شود:

```
git switch -c feature/login
```

پس از اعمال تغییرات، می‌توانید آن‌ها را ثبت و شاخه را به GitHub ارسال کنید:

```
git add .
git commit -m "Add login feature"
git push -u origin feature/login
```

Branch مانند Fork یک مخزن مستقل نیست؛ بلکه مسیر جداگانه‌ای برای توسعه در همان مخزن است و Git از تاریخچهٔ آن استفاده می‌کند.

درخواست ادغام (Pull Requests)

درخواستی برای بررسی تغییرات و ادغام آن‌ها از یک شاخه به شاخه‌ای دیگر است، یا به‌اختصار **PR** یا **Pull Request**.

برای مثال، پس از پیاده‌سازی قابلیت ورود کاربران در شاخهٔ `feature/login`، می‌توانید یک Pull Request ایجاد کنید تا تغییرات پیشنهادی شما برای ادغام در `main` بررسی شوند.

به اعضای تیم اجازه می‌دهد Pull Request:

- تغییرات کد را بررسی کنند.
- نظرها و پیشنهادهای خود را ثبت کنند.
- دربارهٔ روش پیاده‌سازی گفت‌وگو کنند.
- آزمایش‌ها و بررسی‌های خودکار را اجرا کنند.
- پیش از ادغام، درخواست اصلاح تغییرات بدهند.
- پس از آماده شدن تغییرات، آن‌ها را تأیید کنند.

ساخت یک Pull Request

۱. یک Branch ایجاد کنید و تغییرات موردنظر را انجام دهید.

۲. تغییرات خود را Commit کنید.

۳. شاخه را به GitHub Push کنید.

۴. وارد مخزن موردنظر در GitHub شوید.

۵. روی آن کلیک کنید؛ در غیر این صورت وارد تب **Pull requests** شوید و یک درخواست جدید ایجاد کنید، در صورت نمایش گزینهٔ **Compare & pull request**.

۶. شاخهٔ مبدأ و شاخهٔ مقصد را انتخاب کنید.

۷. عنوانی واضح بنویسید و تغییرات انجام‌شده را توضیح دهید.

۸. Pull Request را ایجاد کنید و در صورت نیاز از دیگران درخواست بررسی کنید.

> نکتهٔ مهم: ایجاد Pull Request به‌معنای ادغام خودکار تغییرات نیست. تغییرات باید جداگانه و توسط فردی که مجوز لازم را دارد Merge شوند.

در پروژه‌های متن‌باز اهمیت ویژه‌ای دارد؛ زیرا توسعه‌دهندگان می‌توانند بدون داشتن دسترسی مستقیم برای نوشتن در مخزن اصلی، تغییرات پیشنهادی خود را ارسال کنند Pull Request.

ادغام شاخه‌ها (Merge)

Merge فرایندی است که تغییرات یک شاخه را با شاخهٔ دیگری ادغام می‌کند.

مدیر پروژه می‌تواند شاخهٔ مربوط به قابلیت جدید، Pull Request برای مثال، پس از بررسی و تأیید یک را با `main` ادغام کند.

نمایش ساده‌شدهٔ یک Merge:

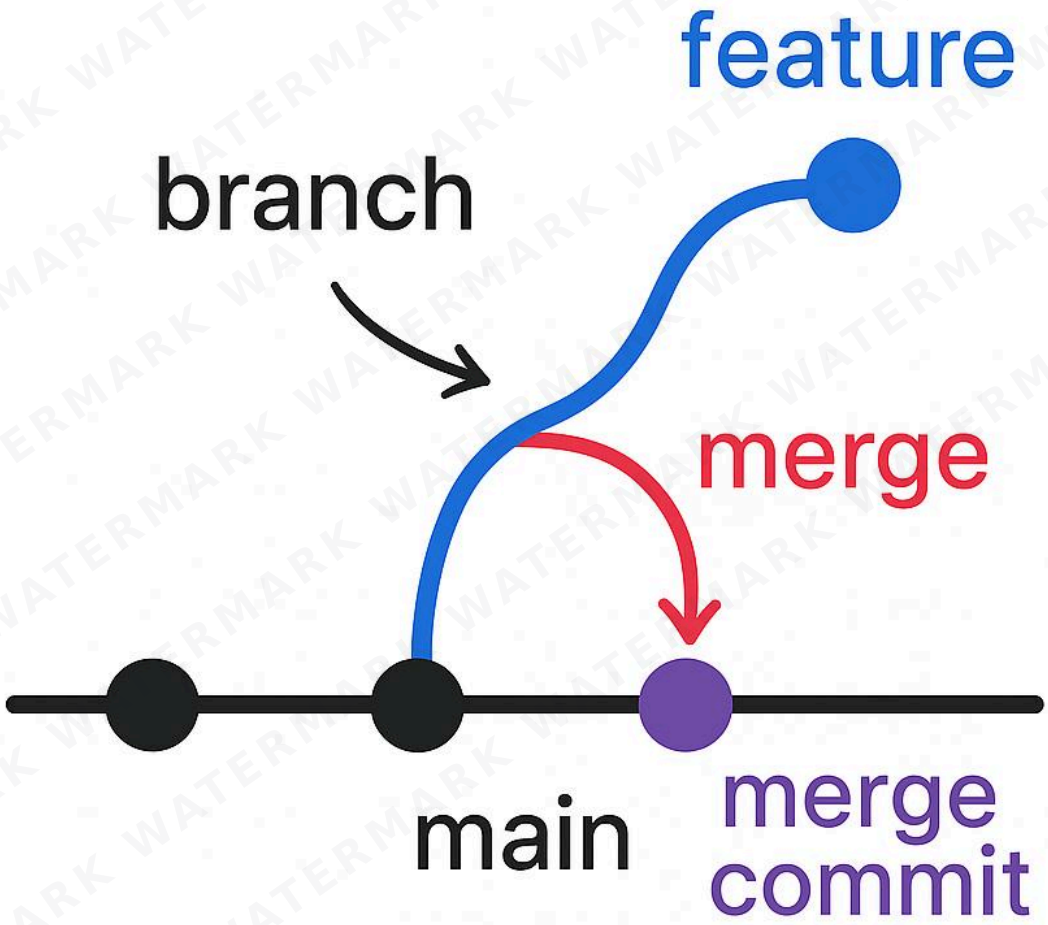
```
قبل از ادغام:
```

```
main:             A --- B --- C
feature/login:          \--- D --- E

پس از ادغام:

main:             A --- B --- C --- M
                              \     /
feature/login:                 D --- E
```

این مثال یک Merge Commit را نشان می‌دهد. Git بسته به تاریخچهٔ مخزن و تنظیمات پروژه می‌تواند از Fast-forward Merge نیز استفاده کند. روش‌های دیگری مانند



روش‌های رایج **Merge** در **GitHub**

Create a Merge Commit

می‌سازد که تاریخچهٔ شاخه‌بندی پروژه را حفظ Merge Commit تغییرات دو شاخه را ادغام می‌کند و یک می‌کند.

Squash and Merge

Commitهای مربوط به Pull Request را به یک Commit تبدیل می‌کند و آن را در شاخهٔ مقصد قرار می‌دهد.

این روش می‌تواند تاریخچهٔ شاخهٔ اصلی را خلاصه‌تر و مرتب‌تر نگه دارد.

Rebase and Merge

Commitهای Pull Request را روی تاریخچهٔ شاخهٔ مقصد بازپخش می‌کند، بدون اینکه Merge Commit جداگانه‌ای ایجاد شود.

این روش می‌تواند تاریخچه‌ای خطی ایجاد کند؛ هرچند شناسهٔ Commitهای حاصل ممکن است با شناسه‌های قبلی متفاوت باشد.

انتخاب روش مناسب به شیوهٔ توسعه، سیاست‌های تیم و دستورالعمل‌های مشارکت در پروژه بستگی دارد.

فورک (Fork)

ایجاد می‌شود GitHub نسخه‌ای از یک مخزن است که تحت حساب کاربری یا سازمان دیگری در Fork.

این قابلیت به‌خصوص هنگام مشارکت در پروژه‌های متن‌باز کاربرد دارد؛ زیرا معمولاً برای ایجاد تغییر مستقیم در مخزن اصلی به مجوز نوشتن نیاز دارید.

فرایند **Fork** کردن

فرض کنید می‌خواهید در یک پروژهٔ متن‌باز که توسط فرد دیگری مدیریت می‌شود مشارکت کنید.

۱. وارد صفحهٔ مخزن اصلی در GitHub شوید.

۲. روی گزینهٔ **Fork** کلیک کنید.

۳. حساب کاربری یا سازمان مقصد را انتخاب کنید.

۴. مخزن Forkشده را روی کامپیوتر خود Clone کنید.

۵. یک Branch ایجاد کنید و تغییرات موردنظر را انجام دهید.

۶. تغییرات را Commit کنید و به Fork خود Push کنید.

۷. یک Pull Request ایجاد کنید و پیشنهاد دهید تغییرات شما با مخزن اصلی ادغام شوند.

نمایش ساده‌شدهٔ این فرایند:

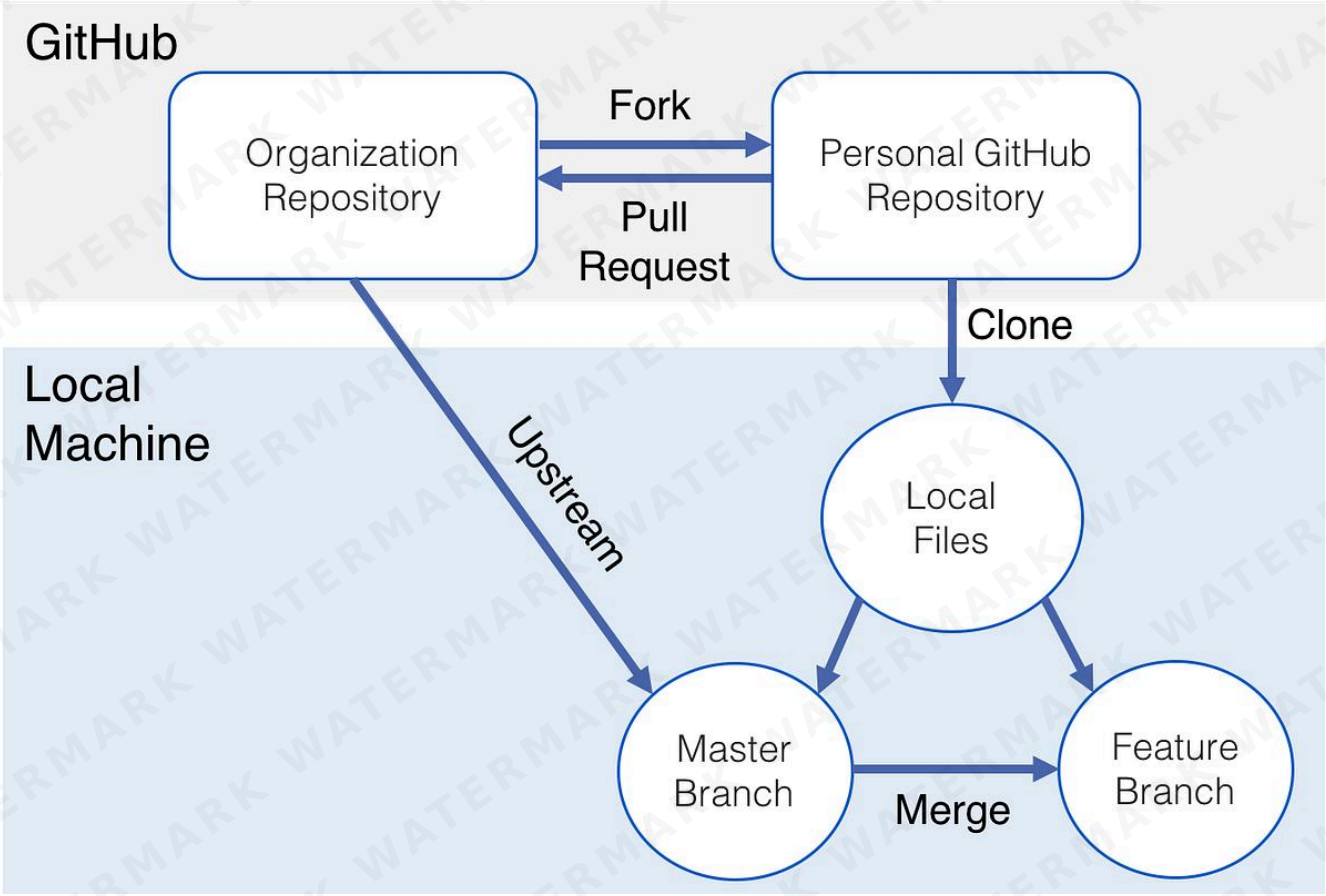
```
مخزن اصلی
    |
    | Fork
    v
فورک شما
    |
    | Clone
    v
کامپیوتر شخصی شما
    |
    | ویرایش، Commit و Push
    v
فورک شما در GitHub
    |
    | Pull Request
    v
مخزن اصلی
```

تفاوت Fork و Branch

ویژگی	Branch	Fork
محل قرارگیری	داخل همان مخزن	مخزنی جداگانه در حساب یا سازمان دیگر
کاربرد رایج	توسعهٔ قابلیت‌ها و کار تیمی	مشارکت در پروژه‌هایی که مالک آن‌ها نیستید
نیاز به دسترسی نوشتن در مخزن اصلی	معمولاً برای Push مستقیم لازم است	برای کردن یک مخزن عمومی Fork معمولاً لازم نیست
مجوزهای مستقل مخزن	خیر	بله
Pull Request	بین شاخه‌های مخزن	به مخزن اصلی Fork از

یک Fork مستقل است؛ اما لزوماً نسخه‌ای نیست که به‌صورت خودکار و دائمی با مخزن اصلی همگام بماند. برای دریافت تغییرات جدید مخزن اصلی، ممکن است لازم باشد آن‌ها را Fetch و Fork خود را به‌روز کنید.

> نکتهٔ مهم: برای Fork کردن یک مخزن عمومی نیازی نیست عضو پروژه باشید. البته سیاست‌های مخزن و سطح دسترسی‌ها ممکن است بر فرایند مشارکت تأثیر بگذارند.



جمع‌بندی

یکی از ابزارهای مهم در توسعهٔ نرم‌افزار مدرن است. یادگیری کار با GitHub، زبان Repository، به شما کمک می‌کند Issues ها و Pull Request، شاخه‌ها، GitHub Desktop نرم‌افزار، Markdown پروژه‌های خود را بهتر مدیریت کنید و با سایر توسعه‌دهندگان همکاری مؤثرتری داشته باشید.

برای ساخت یک پروفایل حرفه‌ای در GitHub:

- مخازن خود را مرتب و سازمان‌دهی کنید.
- برای پروژه‌ها README‌های واضح و کاربردی بنویسید.
- برای پروژه‌های خود مجوز نرم‌افزاری مناسبی انتخاب کنید.
- از `.gitignore` برای جلوگیری از ثبت فایل‌های غیرضروری و اطلاعات محلی حساس استفاده کنید.
- یک Profile README حرفه‌ای بسازید.
- پورتفولیوی خود را با GitHub Pages منتشر کنید.
- در پروژه‌های متن‌باز مشارکت داشته باشید.
- از Pull Request و Branch برای توسعه و بررسی تغییرات استفاده کنید.

بهترین راه یادگیری **GitHub**، استفادهٔ عملی از آن است. با یک پروژهٔ کوچک شروع کنید، آن را منتشر کنید، به‌مرور بهبود دهید و سپس در پروژه‌های سایر توسعه‌دهندگان مشارکت کنید.

منابع مفید

- وب‌سایت GitHub
- مستندات Git
- مستندات رسمی GitHub
- آموزش‌های تعاملی GitHub Skills
- دانلود GitHub Desktop
- وب‌سایت GitHub Pages
- انتخاب مجوز نرم‌افزاری
- مستندات GitHub Flavored Markdown
- قالب‌های `.gitignore`